

REST Assured API Automation Assessment

Prepared by: Salman Benny Kaliboina

Date: 29 June 26

Overview

This document captures the setup, design decisions, and test results for the Rest Assured automation framework, built to check the required API using GET method.

We went with the Junit and RestAssured assertions because both are different approaches to validate the response coming from given url.

URL: <https://jsonplaceholder.typicode.com/posts/1>

What the framework covers

The suites are organised around the main user journeys:

- Authentication — login, logout, password reset, SSO flows
- User management — creating, editing, and deleting accounts
- Search and filtering — keyword search, advanced filters, pagination
- Checkout — cart, the full checkout sequence, and order confirmation
- API layer — response validation via REST-assured
- Cross-browser — Chrome, Firefox, and Edge

How the layers fit together

There are four main layers. Tests call into page objects, which handle all the browser interaction. Utility classes deal with waits, screenshots, and reading test data. Config is kept in properties files so the same tests can run against dev, QA, or staging without touching any code.

Tech Stack

The choices below are fairly standard for a Java/Selenium project. A few notes on why we landed where we did are underneath the table.

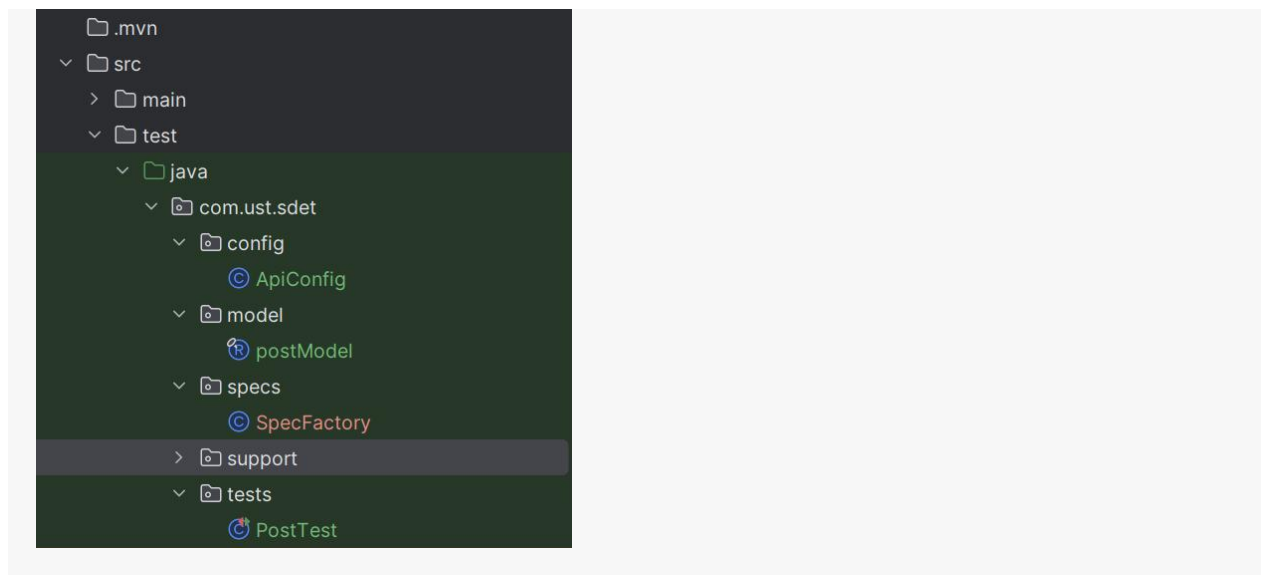
Category	Technology	Version
Language	Java	17 LTS
Assertions	RestAssured, Junit	5.14.4
Test runner	TestNG	7.8.x

Category	Technology	Version
Build	Maven	3.9.x
Reports	Extent Reports	5.x
Logging	Log4j2	2.x
Version control	Git / GitHub	Latest

Folder Structure

The project follows standard Maven layout. Everything under src/main is framework code; everything under src/test is the actual tests and test resources.

Framework source (src/main/java)



Description:

Created a project with separation of code into a structured pattern.

Config: Configuring the basic setup. Setting up the base url, where the execution of code starts.

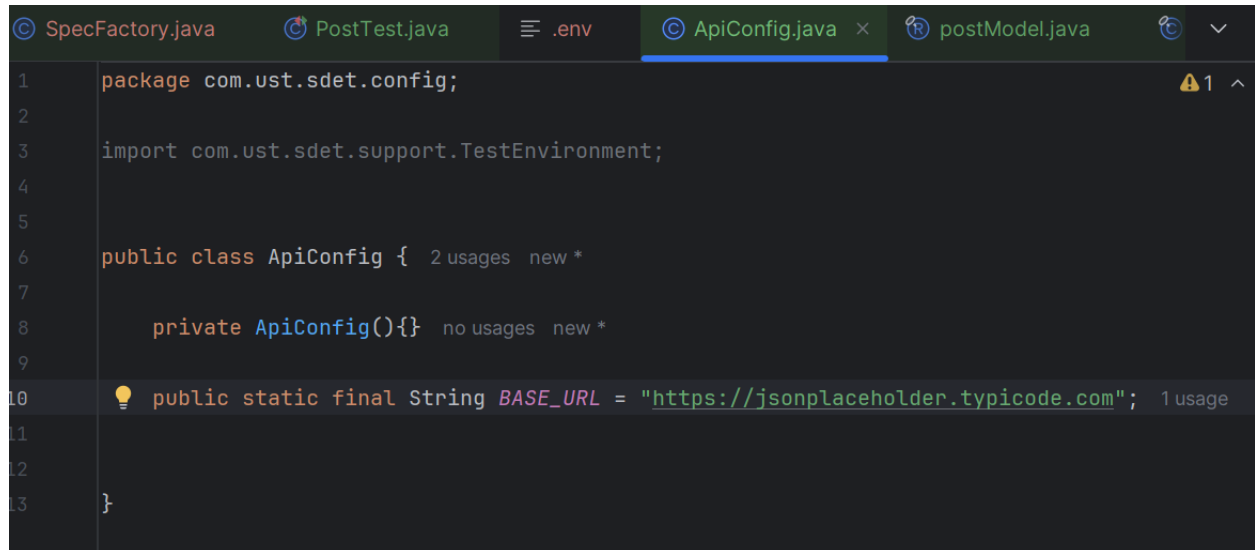
Model: Created records to store the response as a class object. Could have gone for POJO model but records are well structured and efficient.

Specs: Contains the SpecFactory with request and response specifications

Tests: Contains test code with @Test annotation

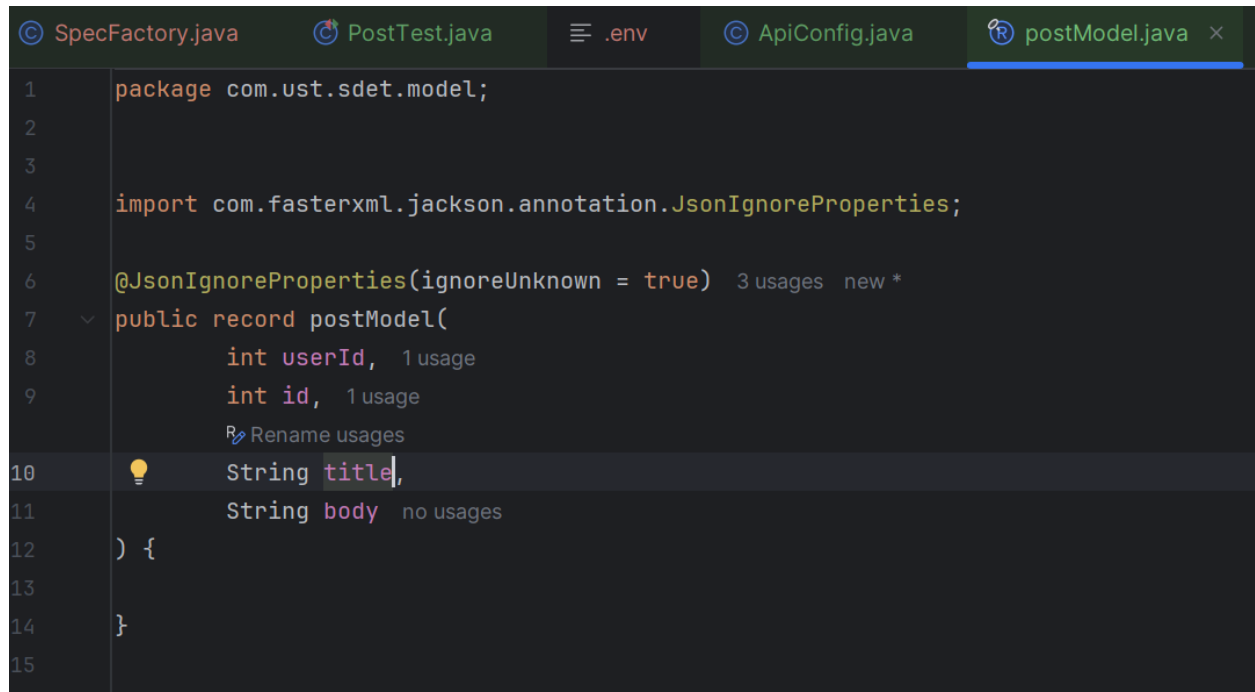
Support: Contains the test environment which access the environment variable defined

ApiConfig:



```
1 package com.ust.sdet.config;
2
3 import com.ust.sdet.support.TestEnvironment;
4
5
6 public class ApiConfig { 2 usages new *
7
8     private ApiConfig(){} no usages new *
9
10    ⚡ public static final String BASE_URL = "https://jsonplaceholder.typicode.com"; 1 usage
11
12
13 }
```

PostModel:



```
1 package com.ust.sdet.model;
2
3
4 import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
5
6 @JsonIgnoreProperties(ignoreUnknown = true) 3 usages new *
7 public record postModel(
8     int userId, 1 usage
9     int id, 1 usage
10    ⚡ String title,
11    String body no usages
12 ) {
13
14 }
15
```

SpecFactory:

```
import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.anyOf;
import static org.hamcrest.Matchers.equalTo;

public class SpecFactory { 2 usages

    public static RequestSpecification reqspec() { no usages
        return new RequestSpecBuilder()
            .setAccept(ContentType.JSON)
            .setContentType(ContentType.JSON)
            .build();
    }

    public static ResponseSpecification resspec(){ no usages
        return new ResponseSpecBuilder()
            .expectStatusCode( expectedStatusCode: 200)
            .expectContentType(ContentType.JSON)
            .build();
    }
}
```

Tests (src/test)

```
public class PostTest{ new *

    @BeforeEach new *
    void setup(){
        RestAssured.baseURI = ApiConfig.BASE_URL;
    }

    @Test new *
    @DisplayName("GET: Validating the details using Junit Assertions")
    void getDetails(){

        postModel pos =
            given() RequestSpecification
                .spec(SpecFactory.reqspec())
                .when()
                .log().all()
                .get(s: "/posts/1") Response
                .then() ValidatableResponse
                .spec(SpecFactory.resspec())
                .extract() ExtractableResponse<Response>
                .as(postModel.class);

        assertEquals( expected: 1,pos.userId());
        assertEquals( expected: 1,pos.id());
        assertTrue(pos.title().contains("sunt aut"));
        assertTrue(pos.body().contains("molestiae"));
```

```

@Test new *
@DisplayName("GET: Validating the details using RestAssured Assertions")
void getDetails1(){

    given() RequestSpecification
        .spec(SpecFactory.reqspect())
        .when()
        .log().all()
        .get(s: "/posts/1") Response
        .then() ValidatableResponse
        .spec(SpecFactory.resspec())
        .body(s: "userId",equalTo(operand: 1))
        .body(s: "id",equalTo(operand: 1))
        .body(s: "title",notNullValue())
        .body(s: "body",notNullValue())
        .body(s: "title",containsString(substring: "sunt aut"))
        .body(s: "body",containsString(substring: "molestiae"))

}

```

Screenshots

Test Results:

Run PostTest x

2 sec 949 ms ✓ 2 tests passed 2 tests total, 2 sec 949 ms

- ✓ GET: Validating the details using Junit Assertions 2 sec 320 ms
- ✓ GET: Validating the details using RestAssu 629 ms

Request method: GET
 Request URI: <https://jsonplaceholder.typicode.com/posts/1>
 Proxy: <none>
 Request params: <none>
 Query params: <none>
 Form params: <none>
 Path params: <none>
 Headers: Accept=application/json, application/javascript, text/javascript, text/json
 Content-Type=application/json
 Cookies: <none>
 Multiparts: <none>

Running the Tests

Prerequisites

- Java 17+ (JDK)
- Maven 3.9+
- Chrome, Firefox, or Edge installed locally
- Git

Commands

mvn clean test

or

mvn test -Dtest=PostTest

Reports

After any run you'll find the logs. Logs are in logs/test-execution.log.

REST Assured API Framework Structure

Framework Components:

- RestAssured with JUnit 5
- RequestSpecification & ResponseSpecification
- Hamcrest Assertions
- POJO Serialization/Deserialization
- Maven, Log4j2, Extent Reports
- GitHub Actions/Jenkins CI

Assertions used

RestAssured:

statusCode(), body(),contentType()

Hamcrest:

equalTo, not, containsString, nullValue, notNullValue,
allOf,.

JUnit 5:

assertEquals, assertNotEquals, assertTrue, assertFalse,